DATA PRE PROCESSING IN PYTHON

The Machine Learning Process

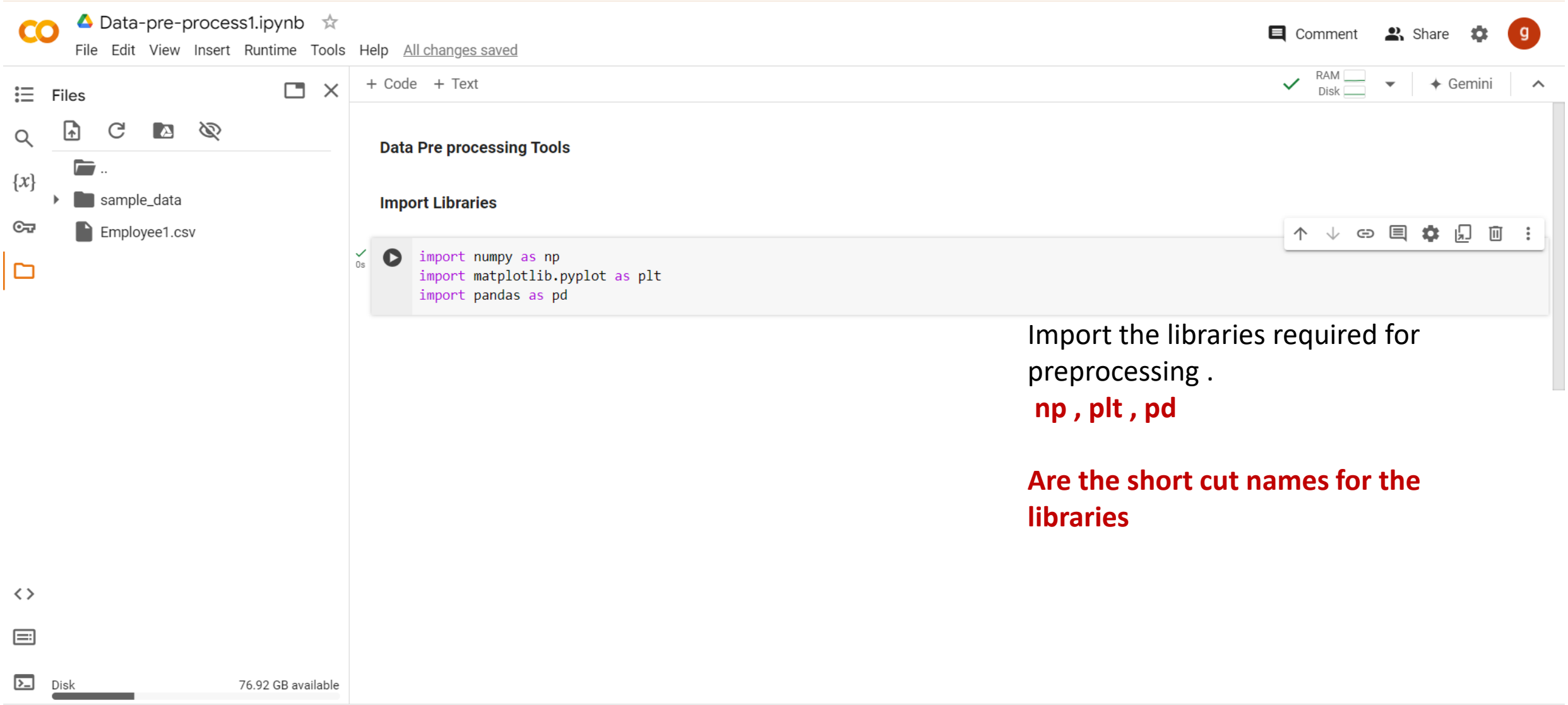
Data Pre-Processing
- Import the data
- Clean the data
- Split into training & test sets

Modelling
- Build the model
- Train the model
- Make predictions

Evaluation
- Calculate performance metrics
- Make a verdict

NumPy will allow us to work with arrays

matplotlib, library that will allow us to plot some **very nice charts**.



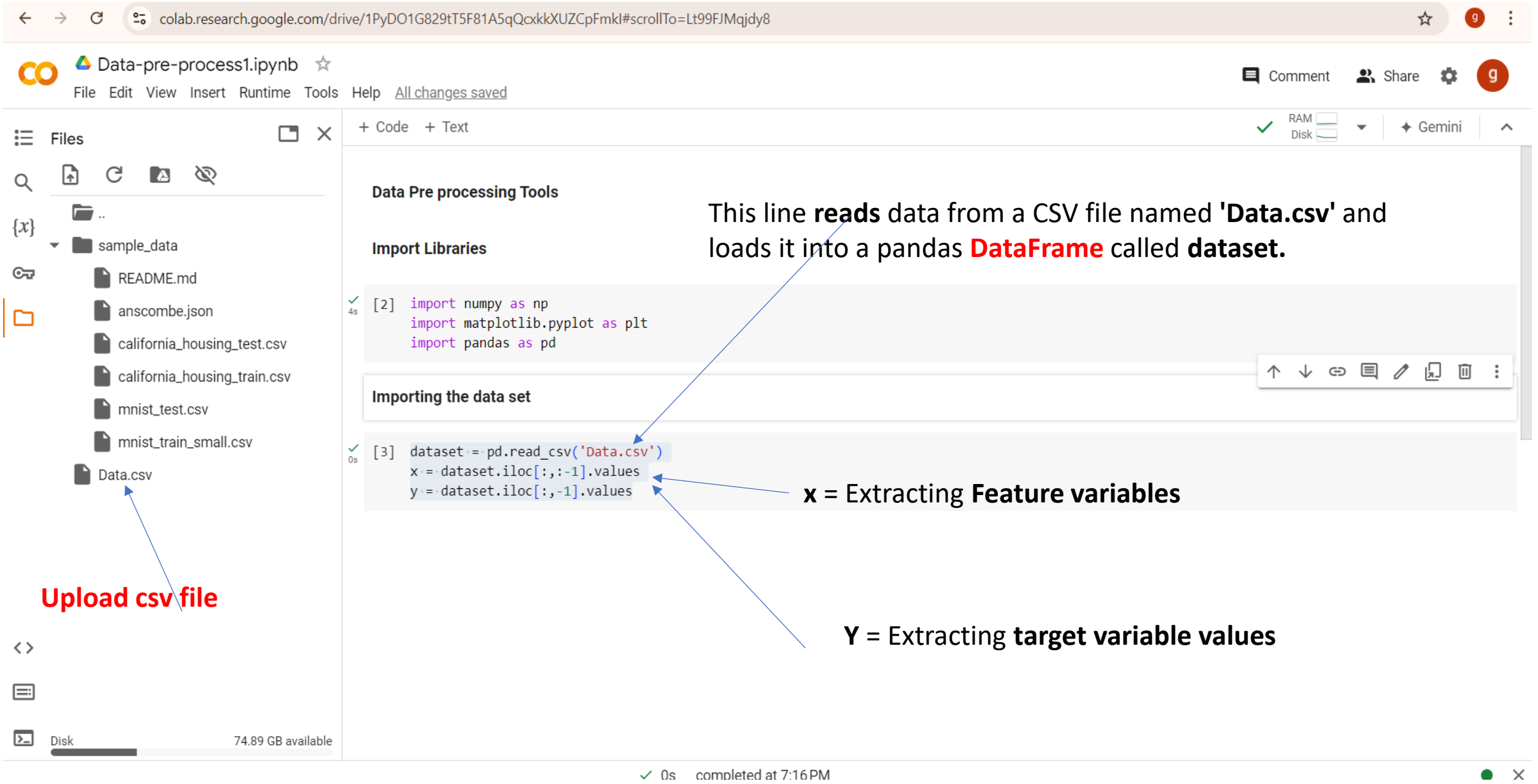
Import the libraries required for preprocessing .

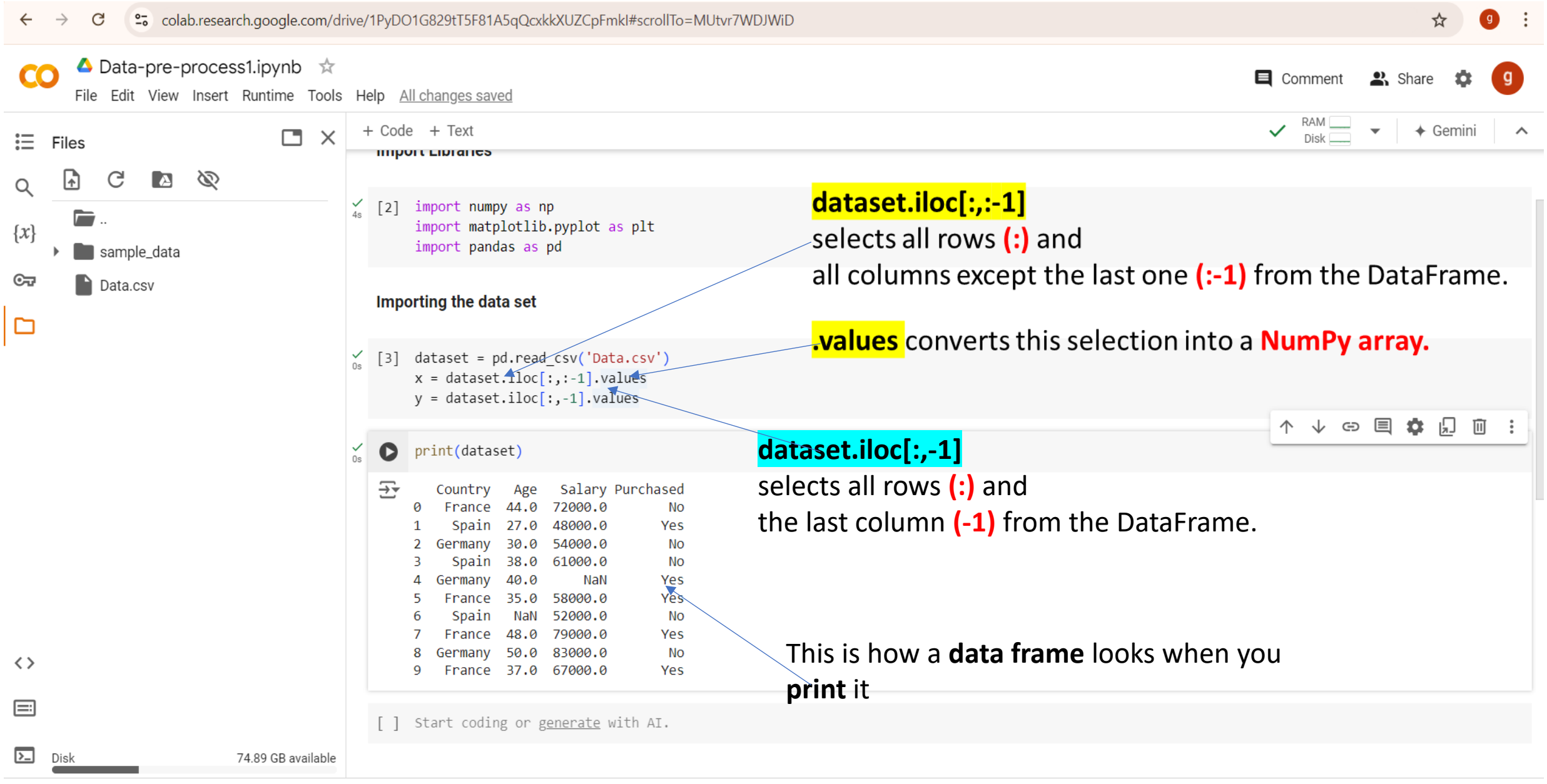
np , plt , pd

Are the short cut names for the libraries

particular **module** called **Pyplot** inside **matplotlib** library and that's the module that allows us to plot **very nice charts**.

pandas, used to create data frames , a two dimensional data structure


colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=Lt99FJMqjdy8
Data-pre-process1.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
Files
sample_data
README.md
anscombe.json
california_housing_test.csv
california_housing_train.csv
mnist_test.csv
mnist_train_small.csv
Data.csv
Disk 74.89 GB available
+ Code + Text
RAM
Disk
Gemini
Data Pre processing Tools
Import Libraries
[2] import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
Importing the data set
[3] dataset = pd.read_csv('Data.csv')
x = dataset.iloc[:,:-1].values
y = dataset.iloc[:,-1].values
This line reads data from a CSV file named 'Data.csv' and loads it into a pandas DataFrame called dataset.
x = Extracting Feature variables
Y = Extracting target variable values
Upload csv file
0s completed at 7:16 PM


colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=MUtvr7WDJWiD
Data-pre-process1.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
Files
+ Code + Text
RAM
Disk
Gemini
..
sample_data
Data.csv
Import Libraries
[2] import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
Importing the data set
[3] dataset = pd.read_csv('Data.csv')
x = dataset.iloc[:,:-1].values
y = dataset.iloc[:,-1].values
print(dataset)
Country Age Salary Purchased
0 France 44.0 72000.0 No
1 Spain 27.0 48000.0 Yes
2 Germany 30.0 54000.0 No
3 Spain 38.0 61000.0 No
4 Germany 40.0 NaN Yes
5 France 35.0 58000.0 Yes
6 Spain NaN 52000.0 No
7 France 48.0 79000.0 Yes
8 Germany 50.0 83000.0 No
9 France 37.0 67000.0 Yes
[ ] Start coding or generate with AI.
Disk 74.89 GB available
Connected to Python 3 Google Compute Engine backend
dataset.iloc[:,:-1]
selects all rows (:) and
all columns except the last one (:-1) from the DataFrame.
.values converts this selection into a NumPy array.
dataset.iloc[:,-1]
selects all rows (:) and
the last column (-1) from the DataFrame.
This is how a data frame looks when you
print it

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=z402EAgvQyrE

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk Gemini

x will thus be a 2D array (or matrix)

```
[6]  print(x)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

For **a DataFrame x**, **.values** returns a **2D NumPy array**

.

Double-click (or enter) to edit

Double-click (or enter) to edit

Y will thus be a 1D array

For a Series **y** , **.values** returns a 1D NumPy array

```
[7]  print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

0s completed at 8:04 PM

Data-pre-process1.ipynb - Cola

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=z402EAgvQyrE

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

Data.csv

```
[6]  ['France' 35.0 58000.0]
     ['Spain' nan 52000.0]
     ['France' 48.0 79000.0]
     ['Germany' 50.0 83000.0]
     ['France' 37.0 67000.0]]
```

Double-click (or enter) to edit

Double-click (or enter) to edit

Y will thus be a 1D array

```
[7]  print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Double click the dataset.
To open the file

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Here we can see the dataset having some **missing data**

Disk 74.89 GB available

0s completed at 8:04 PM

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

Files

..
sample_data
Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk

Gemini

Y will thus be a 1D array

```
print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

0s completed at 9:04 PM

Files

..

sample_data

Data.csv

Disk 74.89 GB available

Y will thus be a 1D array

```
print(y)
```

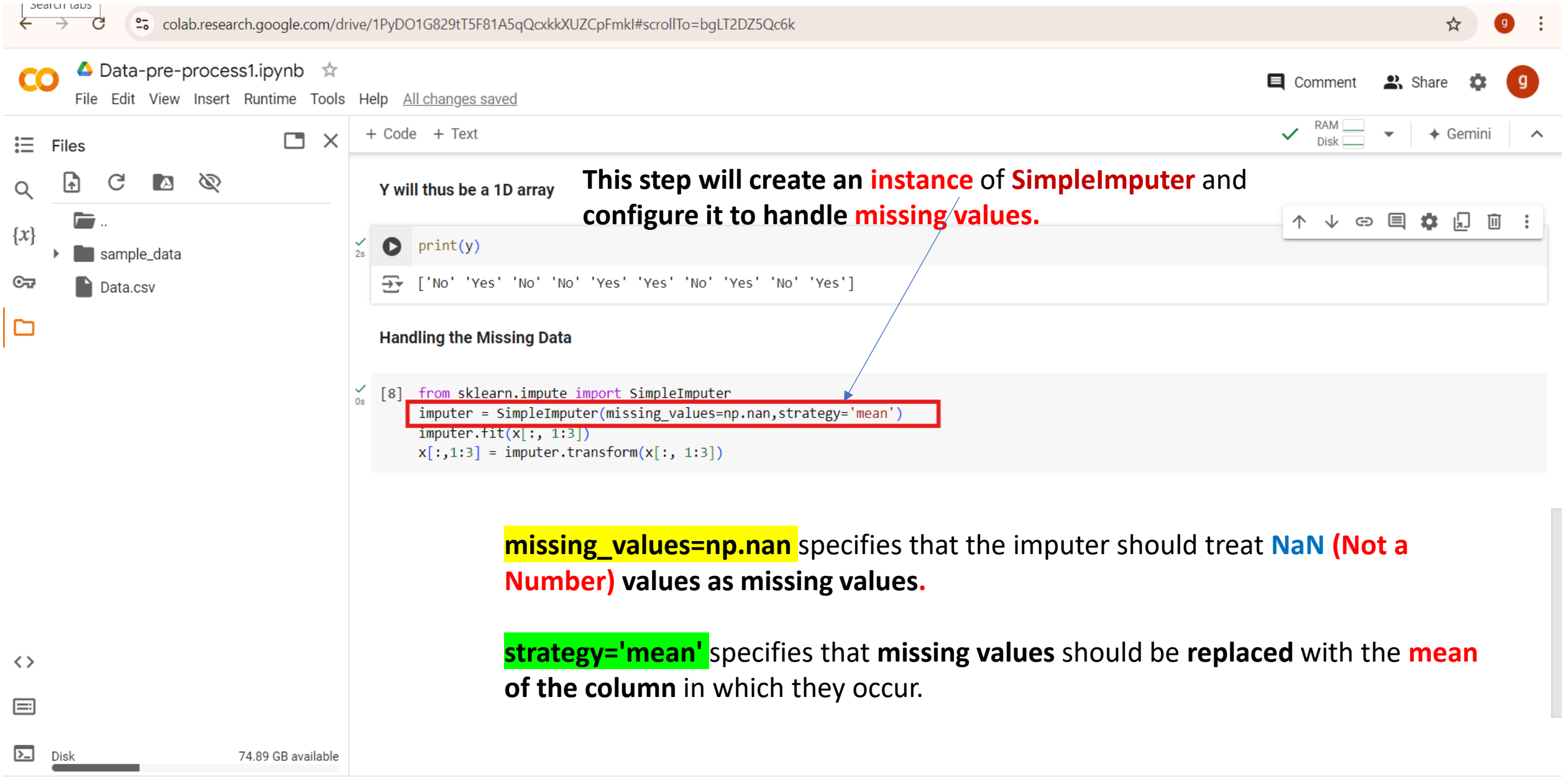
```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

This line imports the **SimpleImputer** class from the **sklearn.impute** module.

SimpleImputer is used for handling missing values in a dataset by imputation.


colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=bgLT2DZ5Qc6k
Data-pre-process1.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
Files
sample_data
Data.csv
+ Code + Text
RAM
Disk
Gemini
Y will thus be a 1D array
This step will create an instance of SimpleImputer and configure it to handle missing values.
print(y)
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
Handling the Missing Data
[8] from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
imputer.fit(x[:, 1:3])
x[:,1:3] = imputer.transform(x[:, 1:3])
missing_values=np.nan specifies that the imputer should treat NaN (Not a Number) values as missing values.
strategy='mean' specifies that missing values should be replaced with the mean of the column in which they occur.
Disk 74.89 GB available
0s completed at 9:04 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=bgLT2DZ5Qc6k

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk Gemini

This step calculates the **mean** of each of the **selected columns**, which will be used to **replace missing values.**

Y will thus be a 1D array

```
print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

fit() method **computes** the **statistics** needed for **imputation.**

x[:, 1:3] selects all rows **(:)** and columns from index **1 to 2 (i.e., columns 1 and 2, excluding column 3).**

0s completed at 9:04 PM

x = [[10, 20, 30, 40],
[50, 60, 70, 80],
[90, 100, 110, 120]]

Slicing operation in Python

0	1	2	3
10	20	30	40
50	60	70	80
90	100	110	120

Slicing with x[:, 1:3]

: (Row Slice)
The colon **:** means **"select all rows."**

1:3 (Column Slice)
1:3 specifies a range of columns to select.
The starting index is 1 and the ending index is 3 (but the ending index is **exclusive in Python slicing**).

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=bgLT2DZ5Qc6k

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM

Gemini

The **transform()** method applies the imputation process to the data.

Y will thus be a 1D array

```
print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

imputer.transform(x[:, 1:3]) uses the **means** computed during the **fit()** step to replace **missing values (NaN)** in the subset **x[:, 1:3].**

This method returns a new array where missing values are filled with the column means.

Connected to Python 3 Google Compute Engine backend

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=bgLT2DZ5Qc6k

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk Gemini

Assigning Transformed Data Back to Original Array

Y will thus be a 1D array

```
print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

x[:, 1:3] = imputer.transform(x[:, 1:3]) updates the original array **x** with the imputed values.

Connected to Python 3 Google Compute Engine backend

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk

Gemini

```
[7] print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling the Missing Data

```
[8] from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
    imputer.fit(x[:, 1:3])
    x[:,1:3] = imputer.transform(x[:, 1:3])
```

```
print(x)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

Now we can see the Missing values is replaced.

0s completed at 10:33 PM

Encoding Categorical Data

Many machine learning algorithms, especially those that rely on **mathematical operations** (**like linear regression or neural networks**), can only handle **numerical input**.

Numerical data can be processed more efficiently than categorical data, leading to faster computations and reduced memory usage.

Types of Encoding:

1.Label Encoding:

Assigns a unique integer to each category.

If you have a feature "**Color**" with categories **["Red", "Green", "Blue"],** label encoding might convert these to [0, 1, 2].

.....

2.One Hot Encoding

Creates a binary column for each category, with a value of **1** indicating the presence of that category and **0** indicating its absence.

Example :

House ID	Neighborhood
1	Downtown
2	Suburb
3	Countryside
4	Downtown
5	Suburb

One-Hot Encoding Process:

Identify Categories: The **"Neighborhood"** feature has three distinct categories: **"Downtown," "Suburb," and "Countryside."**

Create Binary Columns: For each category, create a new binary column:

Neighborhood_Downtown
Neighborhood_Suburb
Neighborhood_Countryside

Convert Categories to Binary Values: For each house, put a **1** in the column corresponding to its neighborhood and **0** in the others.

.....

Encoded Data

House ID	Neighborhood_Downtown	Neighborhood_Suburb	Neighborhood_Countryside
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0
5	0	1	0

3. Binary encoding

Two main steps:

Assign an Integer to Each Category: Each unique category is assigned a **unique integer value.**

Convert Integers to Binary Code: Convert **these integers into binary code**. Then, represent **each binary digit (bit) as a separate column.**

Encode Categories

City	Integer	Binary	City_Bit1	City_Bit2
New York	0	00	0	0
Los Angeles	1	01	0	1
Chicago	2	10	1	0
Houston	3	11	1	1

Binary Encoded Data

City	City_Bit1	City_Bit2
New York	0	0
Los Angeles	0	1
Chicago	1	0
Houston	1	1

Compared to **one-hot encoding**, which creates a **separate column for each category**, **binary encoding** uses **fewer columns**, which can be more efficient in terms of memory and processing, especially for features with many categories.

For instance, if a feature has **256 categories**, one-hot encoding would require **256 columns**, whereas **binary encoding would only require 8 columns** (since 256 in binary requires 8 bits).

Files

..

sample_data

Data.csv

```
['Germany' 40.0 63777.7777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

Encoding Independent Variable

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    x = np.array(ct.fit_transform(x))
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk Gemini

```
['Germany' 40.0 63777.77777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

Encoding Independent Variable

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    x = np.array(ct.fit_transform(x))
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

ColumnTransformer is a tool that allows you to apply **different transformations to different columns of your dataset.**

Search

0s completed at 8:27 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

Data.csv

```
['Germany' 40.0 63777.7777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

Encoding Independent Variable

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    x = np.array(ct.fit_transform(x))
```

OneHotEncoder is used to convert **categorical variables** into a **one-hot numeric array.**

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Disk 74.89 GB available

Search

0s completed at 8:27 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5gOcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

[0] specifies that the transformation should be applied to the column at index 0 (the first column).

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

Files

- ..
- sample_data
- Data.csv

Disk 74.89 GB available

+ Code + Text

RAM Disk Gemini

```
['Germany' 40.0 63777.77777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

'encoder' is just a name for this specific transformation.

Encoding Independent Variable

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    X = np.array(ct.fit_transform(X))
```

OneHotEncoder() is the actual transformation function to apply.

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Search

0s completed at 8:27 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

Data.csv

Disk 74.89 GB available

```
['Germany' 40.0 63777.77777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

Encoding Independent Variable

It's a argument

It's a argument

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    X = np.array(ct.fit_transform(X))
```

This argument is a list of tuples

remainder='passthrough' indicates that **columns** not specified in the **transformers list** should be **left unchanged**.

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Search

0s completed at 8:27 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=lh_XLKB1ymTv

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

Data.csv

```
['Germany' 40.0 63777.7777777778]
['France' 35.0 58000.0]
['Spain' 38.77777777777778 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
```

Encoding categorical data

Encoding Independent Variable

```
[9] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder='passthrough')
    x = np.array(ct.fit_transform(x))
```

ct.fit_transform(x) applies the transformation to the dataset **x**.

np.array(...) converts the result **back into a NumPy array**, **though it's already a NumPy array by default**.

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Disk 74.89 GB available

Search

0s completed at 8:27 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=9sKVmjD73zFQ

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

Data.csv

Disk 74.89 GB available

['France' 37.0 67000.0]]

Encoding categorical data

Encoding Independent Variable

```
[42] from sklearn.compose import ColumnTransformer
     from sklearn.preprocessing import OneHotEncoder
     ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='passthrough')
     x = np.array(ct.fit_transform(x))
```

```
print(x)
```

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

France is encoded as **one zero and zero**
Spain is encoded as a vector **of zero, zero and one**
Germany is encoded as a vector of **zero, one and zero.**

Connected to Python 3 Google Compute Engine backend

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=9sKVmjD73zFQ

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

+ Code + Text

RAM Disk Gemini

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

Data.csv

Disk 74.90 GB available

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

Encoding Dependent Variable

```
[10] from sklearn.preprocessing import LabelEncoder
     le = LabelEncoder()
     y = le.fit_transform(y)
```

```
[11] print(y)
```

```
[0 1 0 0 1 1 0 1 0 1]
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

0s completed at 7:34 PM

LabelEncoder is a tool used for encoding **categorical labels** into **numerical values**.

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=rJNaV_O6cILC

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

..
sample_data
README.md
anscombe.json
california_housing_test.csv
california_housing_train.csv
mnist_test.csv
mnist_train_small.csv
Data.csv

Disk 74.90 GB available

```
[9] [[1.0 0.0 0.0 44.0 72000.0]
     [0.0 0.0 1.0 27.0 48000.0]
     [0.0 1.0 0.0 30.0 54000.0]
     [0.0 0.0 1.0 38.0 61000.0]
     [0.0 1.0 0.0 40.0 63777.77777777778]
     [1.0 0.0 0.0 35.0 58000.0]
     [0.0 0.0 1.0 38.77777777777778 52000.0]
     [1.0 0.0 0.0 48.0 79000.0]
     [0.0 1.0 0.0 50.0 83000.0]
     [1.0 0.0 0.0 37.0 67000.0]]
```

Encoding Dependent Variable

```
[10] from sklearn.preprocessing import LabelEncoder
     le = LabelEncoder()
     y = le.fit_transform(y)
```

```
print(y)
```

```
[0 1 0 0 1 1 0 1 0 1]
```

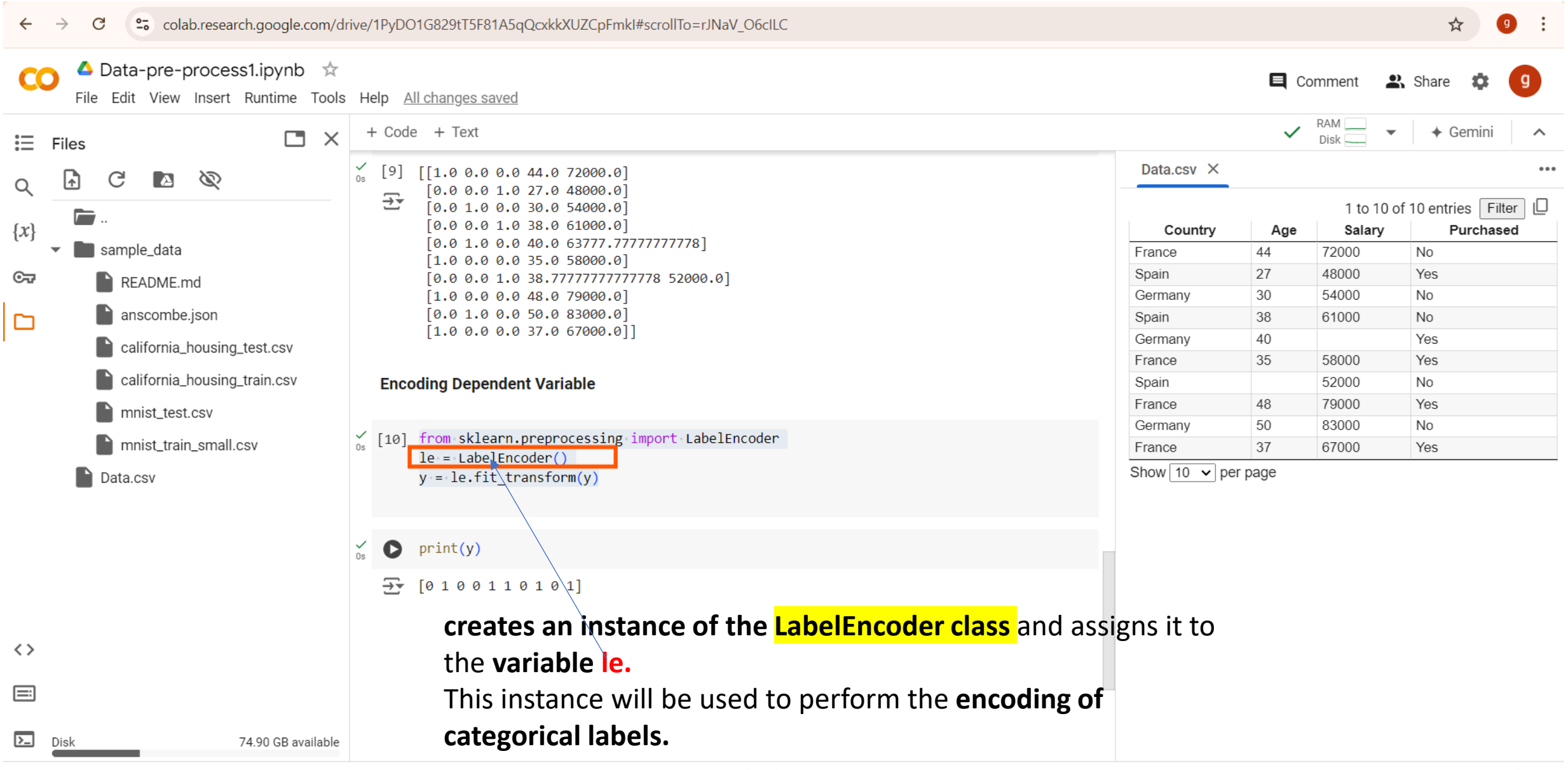
Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

0s completed at 7:34 PM


colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=rJNaV_O6cILC
Data-pre-process1.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
Files
+ Code + Text
RAM
Disk
Gemini
..
sample_data
README.md
anscombe.json
california_housing_test.csv
california_housing_train.csv
mnist_test.csv
mnist_train_small.csv
Data.csv
Disk 74.90 GB available
[9] [[1.0 0.0 0.0 44.0 72000.0]
[0.0 0.0 1.0 27.0 48000.0]
[0.0 1.0 0.0 30.0 54000.0]
[0.0 0.0 1.0 38.0 61000.0]
[0.0 1.0 0.0 40.0 63777.77777777778]
[1.0 0.0 0.0 35.0 58000.0]
[0.0 0.0 1.0 38.77777777777778 52000.0]
[1.0 0.0 0.0 48.0 79000.0]
[0.0 1.0 0.0 50.0 83000.0]
[1.0 0.0 0.0 37.0 67000.0]]
Encoding Dependent Variable
[10] from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
print(y)
[0 1 0 0 1 1 0 1 0 1]
creates an instance of the LabelEncoder class and assigns it to the variable le.
This instance will be used to perform the encoding of categorical labels.
Data.csv
1 to 10 of 10 entries Filter
Country Age Salary Purchased
France 44 72000 No
Spain 27 48000 Yes
Germany 30 54000 No
Spain 38 61000 No
Germany 40 Yes
France 35 58000 Yes
Spain 52000 No
France 48 79000 Yes
Germany 50 83000 No
France 37 67000 Yes
Show 10 per page
0s completed at 7:34 PM

This line performs two operations on the data y

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=rJNaV_O6cILC

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

RAM Disk Gemini

Files

- ..
- sample_data
  - README.md
  - anscombe.json
  - california_housing_test.csv
  - california_housing_train.csv
  - mnist_test.csv
  - mnist_train_small.csv
- Data.csv

Disk 74.90 GB available

```
[9] [[1.0 0.0 0.0 44.0 72000.0]
     [0.0 0.0 1.0 27.0 48000.0]
     [0.0 1.0 0.0 30.0 54000.0]
     [0.0 0.0 1.0 38.0 61000.0]
     [0.0 1.0 0.0 40.0 63777.77777777778]
     [1.0 0.0 0.0 35.0 58000.0]
     [0.0 0.0 1.0 38.77777777777778 52000.0]
     [1.0 0.0 0.0 48.0 79000.0]
     [0.0 1.0 0.0 50.0 83000.0]
     [1.0 0.0 0.0 37.0 67000.0]]
```

Encoding Dependent Variable

```
[10] from sklearn.preprocessing import LabelEncoder
     le = LabelEncoder()
     y = le.fit_transform(y)
```

```
print(y)
```

```
[0 1 0 0 1 1 0 1 0 1]
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

Fit: The **fit()** method analyzes the unique values in the input data **y** and creates a mapping from these **unique labels to integers**.

This mapping is stored within **the Label Encoder** instance.**(le)**

Transform: The **transform()** method uses the mapping created by **fit()** to convert the original **categorical labels** in **y** into numerical values (integers).

0s completed at 7:34 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=scezgA6Wk4mt

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

Data.csv

Disk 74.90 GB available

+ Code + Text

RAM Disk Gemini

```
[0 1 0 0 1 1 0 1 0 1]
```

Split the data set into Training set and Test set

```
[12] from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(x,y,test_size =0.2,random_state=1)
```

```
[13] print(x_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[14] print(x_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[15] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[16] print(y_test)
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

0s completed at 8:13 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=scezgA6Wk4mt

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

This line imports the **train_test_split** function from the **sklearn.model_selection** module.

This function is commonly used to **split data into training and testing sets.**

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

Data.csv

Disk 74.90 GB available

+ Code + Text

RAM Disk Gemini

```
[0 1 0 0 1 1 0 1 0 1]
```

Split the data set into Training set and Test set

```
[12] from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(x,y,test_size =0.2,random_state=1)
```

```
[13] print(x_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[14] print(x_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[15] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[16] print(y_test)
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

✓ 0s completed at 8:13 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=scezgA6Wk4mt

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

Data.csv

Disk 74.90 GB available

+ Code + Text

RAM Disk Gemini

```
[0 1 0 0 1 1 0 1 0 1]
```

Split the data set into Training set and Test set

```
[12] from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(x,y,test_size =0.2,random_state=1)
```

```
[13] print(x_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[14] print(x_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[15] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[16] print(y_test)
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

x_train: The subset of the feature data **x** used for training the model.
x_test: The subset of the feature data **x** used for testing the model.
y_train: The subset of the target labels **y** corresponding to **x_train**.
y_test: The subset of the target labels **y** corresponding **to x_test.**

0s completed at 8:13 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=scezgA6Wk4mt

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share g

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

Data.csv

Disk 74.90 GB available

+ Code + Text

RAM Disk Gemini

```
[0 1 0 0 1 1 0 1 0 1]
```

Split the data set into Training set and Test set

```
[12] from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(x,y,test_size =0.2,random_state=1)
```

```
[13] print(x_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[14] print(x_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[15] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[16] print(y_test)
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

x:Represents the **feature data (input variables)** that you want to split.

y:Represents the **target labels (output variables)** that you want to split, **corresponding to x**.

0s completed at 8:13 PM

colab.research.google.com/drive/1PyDO1G829tT5F81A5qQcxkkXUZCpFmkI#scrollTo=scezgA6Wk4mt

Data-pre-process1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

test_size=0.2:This parameter specifies the proportion of the dataset to include in the test split.

Here, **0.2** means that **20%** of the data will be used for testing, and the remaining **80%** will be used for training.

+ Code + Text

Gemini

Files

- ..
- sample_data
  - README.md
  - anscombe.json
  - california_housing_test.csv
  - california_housing_train.csv
  - mnist_test.csv
  - mnist_train_small.csv
- Data.csv

Disk 74.90 GB available

```
[0 1 0 0 1 1 0 1]
```

Split the data set into Training set and Test set

```
[12] from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(x,y,test_size =0.2,random_state=1)
```

```
[13] print(x_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[14] print(x_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[15] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[16] print(y_test)
```

Data.csv

1 to 10 of 10 entries Filter

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

Show 10 per page

By setting **random_state** to a specific integer value (e.g., 1), you ensure that **same random splits** will be produced **every time** you execute the code with that same **random_state** value.

0s completed at 8:13 PM